第二次任务

CMake
一、大致介绍
二、学习内容
三、实践任务
1.完成含有CMakeLists.txt的单位转换器项目
2.温馨提示
四、提交要求
1.GitHub或Gitee仓库
2.语雀文档
五、评分标准
六、温馨提示
LLM
一、关于使用
1. 什么是LLM?
2. 什么是Agent?
2.1. 关于安装
2.2. 关于使用
二、关于实践任务
1. 补全网关
2. 你所需要掌握的
三、关于提交要求
1. GitHub或Gitee仓库
2. 语雀文档
四、评分标准
五、温馨提示

CMake

大家在C++培训时只需要写完代码然后直接点击例如DEV C++等IDE中的运行按钮运行，但实际上代码从C++等编程语言到最后的运行结果还需要经过复杂的编译过程。在实际的项目中，一份.cpp文件中往往有几十甚至上百个头文件和库，像大家过去在IDE中那样进行“一键编译”或者编译时手动配置路径显然十分不现实。而学习CMake就能帮助大家解决上面提到的问题。

温馨提示：在CMake和后面内容的学习中，也许你会碰到许多十分陌生的名词，在搜索这些名词的过程中，也许你会发现AI用另一个你不懂的名词解释了这个词的意思。不要灰心，对于零基础的同学来说入门确实不简单。多做一些小练习去实际使用这些陌生的知识也许能让你更快上手。

一、大致介绍

- 告别长命令
  - 在只有一个 `main.cpp` 时，用 `g++ main.cpp -o app` 来编译它非常方便，但是当你有几十个源文件并且要链接几十个第三方库时，编译命令会变得非常长。而CMake能把这些复杂的编译指令写进脚本里，这样一来再编译时只需要 `cmake` 和 `make` 即可。
- 自动寻找并链接第三方库
  - 举个例子，如果你在代码中需要用opencv进行图像处理，你写了 `#include <opencv2/opencv.hpp>` ，但是编译器并不知道这个头文件在哪里，也不知道opencv的库文件在哪里。CMake提供了专业的寻找模块（例如后面你会学到的 `find_package()` ），你可以直接使用 `find_package(OpenCV)` 定位库的安装路径，并把头文件目录和库文件路径准确无误地传递给编译器，这比自己手动去找路径方便也可靠得多。
- 跨平台
  - CMake能够让同一份源码在不同的操作系统上无需大量修改就能正常编译运行。

二、学习内容

也许你会觉得参考资料中的知识比较杂让你抓不住重点，那么至少先熟练掌握下面这些知识，然后能够自己独立完成实践任务。

- 熟练掌握以下常用的CMake命令并能够编写CMakeLists.txt：
  - 基本命令： `cmake_minimum_required()` 、 `set()` 、 `project()` ；
  - 把源码变成可执行文件或库： `add_executable()` 、 `add_library()` ；
  - `target_link_libraries()` 、 `target_include_directories()` 、 `target_compile_definitions()` ；
  - 查找第三方库： `find_packages()` ；
  - 项目分模块管理： `add_subdirectory()` ；
  - 安装与配置： `install()` ；
- 源码外构建：
  - 理解源码外构建的意义；
  - 掌握 `mkdir build && cd build && cmake .. && make` 的标准流程；
- 能解决路径错误、依赖找不到、版本不兼容等常见错误。
- 参考资料：
  - Search — CMake 4.4.2 Documentation
  - CMake Documentation and Community
  - CMake 入门实战

三、实践任务

1.完成含有CMakeLists.txt的单位转换器项目

- 要求：
  - 代码基本功能：能支持以下三组单位转换：摄氏度与华氏度、米和英尺、磅和千克，用户能通

过交互页面自主选择待转换的单位，输出转换结果；

- 目录结构：（**必须严格按照此结构构建**）

```
UnitConverter/
├── CMakeLists.txt              # 根目录 CMake 构建文件
├── src/
│   └── main.cpp                # 主程序（交互逻辑）
├── unitlib/                    # 转换库
│   ├── CMakeLists.txt          # 库的 CMake 构建文件
│   ├── include/
│   │   └── unitlib.h           # 头文件（函数声明）
│   └── src/
│       └── unitlib.cpp         # 转换函数具体实现
└── build/                      # 编译目录（自行创建，不要提交）
```

- 语雀文档相关：可以记录在编写CMakeLists.txt或构建项目时遇到的问题和解决办法、CMakeLists.txt的关键语句截图和注释等内容。

2.温馨提示

和上周的git一样，实现单位转换非常容易，这个迷你项目本身不需要使用CMake，但是考虑到如果要求大家做一个比较复杂的项目可能反而误导大家把注意力放到其他地方而错过了CMake的入门机会，最终徒增大家的负担。希望大家能理解我们设计实践任务的初衷：除了作为考核和选拔的参考之外，更重要的是想让大家能通过本身并不复杂的实践任务加深对新工具和新知识的理解。这里限制了目录结构甚至要求大家写两个CMakeLists.txt是希望能在小项目中尽可能模拟大家日后会接触到的更加庞大的项目的结构，希望大家能够领会精神并认真完成实践任务中CMakeLists.txt的编写。

四、提交要求

1.GitHub或Gitee仓库

- 仓库名称：`HelloCV`
- 仓库内容：
  - `README.md`：详细说明项目结构、环境配置步骤、使用方法；
  - 所有语雀笔记的链接汇总文档；
  - （可选）关键脚本、配置文件等。

2.语雀文档

- 必须包含**至少2篇语雀笔记链接**（分别对应学习内容和实践任务）；
- 每篇需满足：
  - 内容整洁、有个人理解与整理；
  - 包含必要的截图、代码块、问题分析；
  - 确保链接公开可访问（建议用无登录浏览器自测）

- 禁止将Word/TXT附件上传代替语雀文档

五、评分标准

项目	分值	说明
学习内容语雀文档笔记	25分	内容完整性、逻辑性、可读性、个人思考
实践任务及语雀文档记录	75分	功能实现、文档质量、问题解决能力
总分	100分	

注意：

- 语雀链接无法访问→该篇记0分；
- 仅贴代码无分析→扣除50%以上的分数；
- 未按要求明明仓库→可能影响评分

六、温馨提示

- 语雀是你的个人知识库，请认真整理，便于后续复习；
- 使用AI也是很重要的能力，鼓励大家借助AI进行学习，但是切忌直接用AI完成任务；
- 提前测试链接的可访问性，避免因权限问题导致提交失败
- `README.md` 是他人了解你项目的窗口，请写得清晰专业

LLM

一、关于使用

本关考验你与时俱进的能力。

从本周开始，考核任务(尤其校内赛)将变得相对困难且时间相对紧张，因此使用LLM以及各类coding agent是被允许甚至提倡的。

1. 什么是LLM?

LLM，全称Large Language Model，即大语言模型，是对2022年底GPT-3.5横空出世以后，采用Transformer架构基于大规模文本进行训练并推理的所有AI的统称。

到如今的2026年，这些基于概率补全下文的模型已经拥有了极其出色的数学和代码能力，最顶尖的模型甚至在88小时内攻克了千禧年数学难题之一。

即使我们允许使用LLM来辅助完成接下来的任务，但**保有一定手写代码的能力仍然是极其必要的**。AI

越来越强不是放弃学习古法编程的理由。高水平的架构师和研究员们可以一行代码不写是因为他们本身具有高屋建瓴的能力，而一个缺失基本编程能力的人很难正确、高效地与AI进行沟通，更不用说看懂AI写的代码甚至作出修改了。

培训阶段，我们不对一篇代码的“手写量”作任何要求，但当我们就代码某行作出提问时，请确保你有回答的底气。

2. 什么是Agent?

如果说LLM是大脑，那么Agent就是LLM的手脚。Agent可以自己分析任务，将任务逐步分解，把每一步的输出变成下一步的输入。它会自己找到目录、读写文件。Agent的作用绝不仅仅是省去网页端复制粘贴的功夫那么简单，只有当模型接入Agent，它才可以被认为拥有了开始工作的能力。

对于新手来说，我们推荐下面几个Agent平台：

- TRAE
- Qoder
- Workbuddy

这些平台来自国内，简单易用并且部分可以免费使用。

当然，如果你对harness的使用早已驾轻就熟，你当然可以使用自己最趁手的harness，比如claude code、dsh、codex cli、hermes agent、pi agent.....

以Ubuntu 22.04为例，接下来介绍Qoder的安装与使用：

2.1. 关于安装

首先进入Qoder的官网。



应该会自动识别我们的linux系统，得到一个 `.deb` 格式的安装包。

我们进入到下载目录里(Firefox默认为~/Downloads)：

```bash
cd ~/Downloads
```

使用apt进行安装：

```bash
sudo apt install ./Qoder-CN-linux-amd64.deb
```

然后你可以在终端打开Qoder了。

```bash
qoder-cn
```

按照提示进行一些登录，成功之后应该是这样的：



至此，你已完成了Qoder的安装。

2.2. 关于使用

当我们使用agent进行工作时，常常提示我们需要指定所谓“工作区”。

实际上就是你工作的文件夹，指定之后相当于告诉模型接下来的工作只能在这个文件夹下进行。

对话框的右下角可以选择模型，Auto模式会自动选择，对于Qoder支持的模型而言，判别模型能力最直观的方法就是看右侧模型倍率：



按理来讲应该是倍率越高模型能力越强，但现实就是有些模型就是同时具有更低的价格和更强的能力。~~怎么k3比glm5.3的定价还高阿里真的认真在定价吗。。~~

我十分建议使用 `Deepseek-Flash`，dsv4.1已经很强大了。

注意省着点用哦，我也不知道免费额度够不够。不过如果真用完了可以换个平台。

二、关于实践任务

本关考验你搜集信息并进行学习的能力。

由于需要学习的内容过于杂乱且不成体系，关于LLM的培训只需要完成实践任务并截图记录在语雀中。

1. 补全网关

什么是网关？在本次任务中，网关就是本地程序和外部模型中间的服务，用于将输入整理成便于发送或合乎要求的格式，同时将模型传来的输出整理成便于本地程序接收并存储、调用的格式。

你需要克隆这个仓库，gitee版本在这里。事先声明一下这个仓库是用ai写的，仓库的主要目的是提供一个小的实践任务以便学习，注意学习才是重点。本次任务中，建议大大方方使用agent完成任务。

仓库有两个目录：`/skeleton` 和 `/example`。其中你需要在agent的帮助下补全 `/skeleton`，完成一个与llm进行石头剪刀布的任务。`/example` 是参考解法，目的是通过一个参考答案帮助你理解所需要掌握的学习内容。

你可能有些疑问：我们之前学的都是C++，为什么现在弄了一个完全由Python语言描述的仓库？

事实上，在实际的工作中，Python语言的确有它独有的优势，而本次任务的网关脚本确实在Python语言的舒适区。

这不意味着你需要学习或者掌握Python语言，你只需要和AI交流，明白每个函数实际起到的功能和作用就可以。你需要掌握的内容也将在你与AI交流的过程中逐步被发现。

你的语雀文档需要记录你使用AI进行学习、完成任务的过程。

任务的重点是与AI协作学习，并进行收发消息完成石头剪刀布的目标。达到这一目标是最重要的，因此任务具有较高的自由度，你可以不严格按照 `/skeleton` 进行补全。但要求你必须使用Deepseek官方

API作为上游进行通信。也正因此，请保证你的语雀文档思路清晰、内容全面，且有较多自己的理解，这一文档对打分具有极大的影响。

2. 你所需要掌握的

- **使用Deepseek官方API。这个过程可能需要你充一块钱**
- **与agent协作进行开发；**
- **培养一定的工程思维，例如为大模型返回输出不理想时的各类应对。**

三、关于提交要求

1. GitHub或Gitee仓库

- 仓库名称：`HelloCV`
- 仓库内容：
  - `README.md`：详细说明项目结构、环境配置步骤、使用方法；
  - 所有语雀笔记的链接汇总文档；
  - （可选）关键脚本、配置文件等。

2. 语雀文档

- 必须包含**至少1篇语雀笔记链接**（实践任务）；
- 需满足：
  - 内容整洁、有个人理解与整理；
  - 包含必要的截图、代码块、问题分析；
  - 确保链接公开可访问（建议用无登录浏览器自测）
- 禁止将Word/TXT附件上传代替语雀文档

四、评分标准

项目	分值	说明
语雀文档记录	100分	功能实现、文档质量、问题解决能力
总分	100分	

五、温馨提示

- 语雀是你的个人知识库，请认真整理，便于后续复习；
- 提前测试链接的可访问性，避免因权限问题导致提交失败
- `README.md`是他人了解你项目的窗口，请写得清晰专业